

CPG · 대규모 언어모델 및 CVSS 기반 Java 코드 · 웹 애플리케이션 취약점 진단 및 리포팅 시스템

Automated Java Code and Web Vulnerability Detection & Reporting System
Based on CPG, LLM, and CVSS



202155530 김세한 sehankim@pusan.ac.kr
202155577 이경윤 kylee0293@gmail.com
202355579 전해은 silver807@pusan.ac.kr

지도교수 손준영 교수

과제 요약

본 과제에서는 저장소의 Java 소스코드를 분석하고, 검토가 필요한 파일·행·근거를 웹 보고서로 제공하는 ScanOps를 구현하였다. Joern의 Code Property Graph(CPG) 분석과 Qwen3.8-Max의 파일 의미 검토를 연결하고, 각 분석 결과의 출처를 보존하여 경고를 합쳤다. 개발팀은 Java 전용 서버 연결, API 후보와 호출 문맥 구성, 모델 응답 검사, 긴 파일의 분할과 행 번호 복원, 경고 병합, 분석 실패 처리, 결과 저장·화면 표시를 구현하였다. Joern의 분석 기능과 Qwen의 기반 모델은 외부 도구로 활용하였다.

이와 별도로 고정 탐지 규칙의 누락을 LLM이 제안하는 API 규칙으로 보완할 수 있는지 평가하였다. CPG에서 추출한 API 후보에 대해 Qwen이 입력 함수(source)와 위험 함수(sink)의 역할을 제안하면, 구현한 검증·변환 코드가 이를 검사하여 CPG 분석에 적용한다. 아래 수치는 이 규칙 보강 실험의 결과다.

NIST Juliet Java의 38개 취약점 유형(CWE)에서 개발용 190쌍과 평가용 534쌍을 분리하고, 평가용 취약·안전 코드에 대한 총 1,068개 판정을 비교하였다. 고정 CPG에 LLM 생성 규칙을 추가하자 탐지율은 27.0%에서 34.3%, F1은 0.400에서 0.466으로 증가하였다. 추가로 찾은 취약 판정은 39건이었으나 오탐도 27건 증가하여 정밀도는 77.4%에서 72.6%로 감소하였다. 따라서 결과는 규칙 보강에 따른 탐지 범위 확대를 지지하지만 모든 지표에서의 품질 향상을 뜻하지 않는다.

별도로 Java 분석 API와 백엔드·데이터베이스·화면을 연결하고, 두 파일의 취약·안전 시연을 통해 서비스 연동을 확인하였다. 배포 방식은 CPG 경고와 대상 CWE를 제공하지 않은 LLM 의미 검토 경고의 합집합이며, 위 규칙 보강 실험과 구별된다. 실제 CVE 탐색 평가는 완주하지 못했고 배포 방식의 일반화 성능도 입증하지 못했다. 이 과제에서는 탐지 규칙 보강에 따른 효과와 오탐 증가를 비교하고, 구현한 시스템의 동작과 평가 결과를 코드 및 실행 기록으로 정리하였다.

주요어: Java 취약점 탐지, Code Property Graph, 대규모 언어모델, 규칙 생성, 정적 분석

목 차

과제 요약	i
1 서론	1
1.1 개발 배경	1
1.2 기존 문제점	1
1.3 개발 목표	1
1.4 평가 항목	2
1.5 전체 시스템 범위와 평가 대상	2
1.6 보고서 구성	3
2 관련 기술	3
2.1 Code Property Graph와 Joern	3
2.2 LLM 기반 보안 분석	3
2.3 본 과제의 차별점	4
3 요구조건 및 설계 변경 사항	4
3.1 평가 집합의 확대	5
3.2 동적 규칙과 고정 규칙의 분리	5
3.3 LLM 호출 및 재현성 통제	5
3.4 산학협력 멘토 의견 반영	5
4 시스템 설계 및 구현	6
4.1 사용자가 요청한 코드가 보고서가 되기까지	6
4.2 Java 파일을 전용 분석 서버로 전달하는 기능	7
4.3 LLM에 전달할 API 후보와 주변 코드 구성	8
4.4 LLM이 제안한 규칙을 검사하여 적용하는 기능	8
4.5 긴 파일을 나누어 검토하고 원본 행을 복원하는 기능	8
4.6 두 분석기의 경고를 합치고 출처를 보존하는 기능	9
4.7 분석 실패를 구분하고 결과를 저장·표시하는 기능	10
4.8 직접 구현한 기능과 확인 근거 정리	10
5 실험 방법, 결과 및 논의	10
5.1 데이터 출처, 선택 및 분할	11
5.2 비교군 선택과 문헌 근거	11
5.3 정답, 출력 매핑 및 지표	11
5.4 전체 38 CWE 결과	13
5.5 고정 규칙의 구현 방식별 결과	13
5.6 오프라인 능력 기반 router 후보	14
5.7 실제 배포 엔진과 통합 검증	14
5.8 실제 CVE 탐색 평가와 보고 범위	15
5.9 재현성, 실패 처리와 타당성 위협	15
5.10 평가 결과 요약과 활용 시 고려사항	15
6 구성원별 역할 및 개발 일정	15
6.1 구성원별 역할 및 최종 진척	15
6.2 개발 일정	16
7 현실적 제약 사항 및 대책	16
8 결론 및 향후 개선 계획	17
부록 A. 프롬프트 및 규칙 검증 요약	18
부록 B. 재현 경로와 공유 코드 범위	18
부록 C. 배포 버전과 응답 계약	19
부록 D. 제출 코드와 평가 자료 찾아보기	19
9 참고 문헌	21

1. 서론

1.1. 개발 배경

저장소에 새로운 코드가 추가되면 개발자는 보안상 문제가 있는지 확인하고, 문제가 의심되는 파일과 위치를 찾아 수정해야 한다. 이때 단순히 “취약하다”는 판정만 받으면 어떤 코드를 검토해야 하는지 알기 어렵다. ScanOps는 저장소 코드를 분석하여 취약점 후보의 파일·행·이유를 보여주고, 결과를 웹 보고서로 확인할 수 있게 만든 졸업과제다.

최종 단계에서는 Java 코드 분석을 중심으로 구현하였다. Joern이라는 분석 도구로 코드 구조와 데이터 흐름을 확인하고, Qwen이라는 언어모델로 코드의 의미를 검토하도록 연결했다. 또한 Joern이 사용하는 고정 규칙에서 누락된 API 정보를 LLM이 보완할 수 있는지 별도 실험으로 비교했다. 따라서 이 보고서는 분석 서비스를 어떻게 구현했는지와 규칙 보강으로 탐지 결과가 어떻게 달라졌는지를 구분하여 설명한다.

표 1. 주요 용어 정의

용어	이 과제에서의 의미
CPG / Joern	코드 구조와 데이터 흐름을 그래프로 표현한 정보 / 이를 생성·분석하는 외부 도구
LLM / Qwen	코드 문맥을 해석하는 언어모델 / 이 과제에서 호출하는 외부 모델
CWE	SQL 삽입·스크립트 삽입 등 취약점 유형을 구분하는 번호
경고 (finding)	검토할 파일·행·유형·근거를 담은 분석 결과. 실제 악용 가능성이 확정됐다는 뜻은 아님

1.2. 기존 문제점

기존 접근의 한계는 다음 네 가지로 요약된다.

1. 고정 규칙의 범위 한계: 미등록 프레임워크와 외부 라이브러리 API는 source/sink 목록에서 누락된다.
2. LLM 단독 판단의 과잉 일반화: 코드 전체를 의미적으로 분류할 수 있으나, 정확한 데이터 경로 없이 취약하다고 판단할 수 있다.
3. 평가 표본의 부족: 소수 CVE나 몇 개 예제만으로는 성능을 일반화하기 어렵다.
4. 주 지표 편중: F1만 제시하면 미탐과 오탐의 성격을 분리해 해석하기 어렵다.

본 과제는 38개 CWE, 534개의 취약·안전 쌍 테스트그룹 (paired testcase group; 평가 코드 내부 명칭 family)을 고정하고 bad/good 양쪽을 개별 판정하여 총 1,068개의 균형 평가 단위를 사용했다. Precision, Recall, Specificity, False Positive Rate(FPR), Accuracy, F1, Macro F1, Matthews Correlation Coefficient(MCC)를 함께 보고한다.

1.3. 개발 목표

본 과제의 목표는 모든 코드를 LLM에 맡기는 것이 아니라, CPG로 표현하고 규칙으로 확인할 수 있는 취약 후보는 구조 근거와 함께 판정하고, 고정 규칙이 부족한 영역은 Qwen3.8-Max의 의미 지식으로 보완하는 것이다. 표 2는 핵심 목표와 현재 달성 상태를 정리한다.

표 2. 핵심 목표 대비 최종 달성 현황

	핵심 목표	상태	비고
①	Java 전체 CPG 분석 경로 구축	완료	Joern import, API 후보 추출, taint, 경고 생성
②	Qwen3.8-Max 규칙 생성	완료	exact FQN 기반 source/sink/none 제안 및 결정적 gate
③	4개 비교군 대규모 평가	완료	38 CWE, 534 쌍 테스트그룹, 1,068 판정
④	CPG 강점과 한계 분해	완료	직접 7 CWE, taint 14 CWE, 고정 미구현 17 CWE
⑤	Java 전용 제품 통합	완료	별도 분석 EC2, Java 전용 API 경로, 사이트 결과 저장·표시
⑥	양상블 일반화 성능 검증	미완료	배포 엔진은 동작하나 실제 CVE 우위는 입증하지 못함

1.4. 평가 항목

표 3. 평가 질문과 필요한 비교

구분	질문	근거가 되는 비교
평가 1	동일한 Java 표본에서 각 방식의 전체 탐지 성능은 어떠한가?	네 비교군의 TP, FP, FN, TN과 8개 지표
평가 2	CPG로 직접 표현되는 유형과 taint 유형에서 성능 차이는 왜 생기는가?	matcher 종류로 사전 고정 한 7/14/17 CWE 총화
평가 3	Qwen API 규칙이 고정 CPG의 누락을 보완하는가?	고정 CPG 대 CPG+Qwen, 특히 taint 14 CWE
평가 4	실험용 router와 실제 배포 엔진을 같은 성능으로 볼 수 있는가?	입력·결합 정책·검증 수준의 명시적 구분

1.5. 전체 시스템 범위와 평가 대상

ScanOps는 코드 분석(SAST), 웹 실행 결과 분석(DAST), 위험도 및 수정 안내를 하나의 웹 서비스로 연결한 시스템이다. 최종 정량 평가는 이 가운데 **Java CPG** 분석에 **LLM** 생성 규칙을 추가했을 때 탐지 성능이 어떻게 변하는가에 초점을 둔다. CVSS 기반 위험도 표시와 웹 진단은 전체 시스템의 기능으로 구현되었지만, 이 보고서의 Java 정량 성능 비교 대상에는 포함하지 않는다. 서비스가 끝까지 동작하는지와 탐지 성능이 얼마나 좋은지는 각각 사이트 시연과 정답 데이터 평가로 확인한다.

개발팀이 수행한 핵심 작업은 다음과 같다.

1. 분석 절차 구현: Joern의 API 후보와 코드 문맥을 LLM 입력으로 구성하고, 응답을 후보의 정확한 메서드 식별자에 연결하여 허용된 source/sink 규칙으로 변환하였다.
2. 평가 절차 구현: Juliet 표본 선택·중복 제거·개발/평가 분리, 정답 표지 중립화, 경고의 파일·행·CWE 매핑, 비교 결과 집계를 구현하였다.
3. 결과 해석: 동일 평가집합에서 고정 CPG, 규칙 보강 CPG, Qwen 단독, CodeQL을 비교하고, 현재 고정 규칙의 구현 방식에 따라 효과와 실패를 분해하였다.
4. 서비스 구현: Java 전용 라우팅, 분석 실패 전파, 경고의 출처·위치 보존, 결과 저장과 화면 표시를 연결하였다.

Joern의 CPG 생성·기본 분석 기능, Qwen의 기반 모델, CodeQL, Juliet 데이터, OWASP ZAP은 외부 자산이다. 개발팀은 이러한 외부 도구를 활용하여 분석 절차, 평가 코드, 서비스 연동 기능을 구현하였다. 과거 QLoRA 파인튜닝은 선행 개발 이력으로 구분한다.

1.6. 보고서 구성

2장은 관련 기술, 3장은 초기 설계에서 최종 Java 구조로의 변경, 4장은 실제 코드 중심의 구현 과정, 5장은 실험 방법·결과·해석을 설명한다. 6장은 구성원 역할과 일정, 7장은 제약, 8장은 결론과 후속 개발 계획을 설명한다. 이 구성은 졸업과제의 설계 변경·역할·일정 기록을 보존하면서 평가 항목과 검증 결과를 연결한다.

2. 관련 기술

2.1. Code Property Graph와 Joern

CPG는 소스코드를 구문 구조, 제어 흐름, 호출 관계, 데이터 의존성의 다중 그래프로 표현한다. Joern은 여러 언어의 코드를 CPG로 변환하고 그래프 질의와 taint 분석을 수행하는 오픈소스 플랫폼이다[2]. ScanOps는 Java 소스에 대해 Joern 프론트엔드를 실행하고, 고정 source/sink/sanitizer specification을 질의 스크립트에 전달한다.

CPG의 장점은 경보에 구조적 근거를 연결할 수 있다는 점이다. 예컨대 HTTP 입력이 여러 변수와 내부 메서드를 거쳐 SQL 실행 함수에 도달한다면, 단순히 위험 문자열이 존재한다는 사실을 넘어 source-to-sink 경로를 제시할 수 있다. 반대로 위험 API의 존재나 인자 literal만으로 판단 가능한 직접 오용은 전체 경로 탐색 없이 정확하게 판정할 수 있다.

CodeQL도 Java의 local/global data flow와 taint tracking을 지원하며, 표준 query pack과 model pack 생태계가 성숙한 강력한 대안이다[3, 4]. 본 과제는 “CodeQL보다 Joern이 항상 우수하다”고 주장하지 않는다. 표 4과 같이 분석 엔진의 중간표현과 동적 규칙 삽입을 직접 통제해야 했기 때문에 Joern을 구현 기반으로 선택하고, CodeQL은 독립 비교 기준선으로 사용했다.

표 4. Joern과 CodeQL 비교 및 Joern 선택 근거

비교 항목	Joern	CodeQL
중간표현	AST·CFG·호출·데이터 흐름을 통합한 CPG와 traversal DSL	CodeQL database와 QL data-flow/taint library
규칙 확장	CPG node·edge·method FQN을 직접 질의하고 외부 specification을 주입하기 쉬움	query/model pack으로 정교하게 확장 가능하나 QL 모델 작성이 필요
본 과제 적합성	후보 API와 주변 CPG 사실을 추출해 LLM 입력으로 재사용하기 용이	성숙한 기본 질의의 독립 기준선으로 적합

따라서 구현 기반은 Joern으로 선택하되, 성능 평가는 CodeQL을 함께 실행하여 특정 분석기에 유리한 결과만 남기지 않았다.

2.2. LLM 기반 보안 분석

LLM은 함수 이름, 라이브러리 문서, 호출 문맥에서 API의 보안 역할을 추론할 수 있지만 저장소 전체의 경로를 일관되게 추적하기 어렵다. 관련 문헌에서는 이 한계를 “LLM 단독 분류”가 아니라 프로그램 분석과의 역할 분담으로 다룬다. IRIS는 Java 프로젝트와 대상 CWE를 입력받아 LLM으로 source/sink specification을 생성하고 CodeQL 경로를 만든 뒤, LLM 문맥 분석으로 경보를 다시 거른다[5]. LLMDFFA는 source/sink 추출, 함수별 data-flow 요약, 경로 가능성 검증으로 문제를 분해하고 Juliet에서 전체 성능과 단계별 성능을 각각 측정했다[7]. GPTScan은 LLM을 취약 여부의 최종 판정기보다 코드의 시나리오·속성 이해에 사용하고 정적 분석으로 후보를 확인했다[8]. LLMxCPG는 CPG 기반 slice로 LLM 입력을 줄이면서 취약 관련 구조를 보존하는 방향을 제시했다[9]. RepoAudit은 저장소를 주요 기반으로 탐색하고 validator와 cache를 제거한 ablation까지 보고하여, 정확도뿐 아니라 경로 검증과 비용을 독립 변수로 다뤘다[10].

이 문헌들의 공통점은 동일 데이터에서 정적 분석 기준선, LLM 결합 방식, 구성요소 제거 실험을 비교하여 어느 요소가 효과를 만들었는지 분리한다는 점이다. ScanOps도 이를 따라 모델 종류 경쟁을 주 비교 목적으로 두지 않고 Qwen3.8-Max를 고정한 채 고정 CPG, Qwen 규칙을 더한 CPG, Qwen 소스

단독, 외부 정적 분석기 CodeQL을 같은 manifest로 비교했다. GPT·Claude를 추가하지 않은 이유는 이들을 포함하면 “CPG 결합 효과”와 “기본 모델 차이”가 동시에 바뀌기 때문이다. 다만 IRIS가 여러 공개·비공개 모델을 별도 평가 항목으로 비교한 것처럼, ScanOps도 향후 동일 프롬프트의 다중 모델 민감도 분석이 필요하다.

2.3. 본 과제의 차별점

ScanOps는 규칙 생성 단계에서 CPG 전체를 LLM에 전달하지 않고, 외부 호출과 내부 파라미터에서 추출한 요약 정보를 입력으로 사용한다. LLM의 제안도 바로 탐지 규칙으로 적용하지 않고 exact FQN 채부착, JSON schema 검사, role whitelist, resolved·high-confidence gate를 거치도록 하였다.

또한 실험용 규칙 생성 경로와 실제 배포의 파일 단위 의미 검토를 분리하였다. 배포에서는 CPG 경고를 유지하면서 high-confidence Qwen 경고만 추가하고, 성능 결과는 전체 38 CWE와 직접 API/인자, source-to-sink taint, 고정 규칙 미구현 영역으로 나누어 해석하였다. Java 요청만 새 EC2의 Joern+Qwen 엔진으로 보내고 기존 언어는 이전 경로에 유지하여 서비스 전환에 따른 영향을 줄였다.

IRIS와 ScanOps는 LLM으로 source/sink 사양을 보완한다는 문제의식은 같지만 정적 분석 기반(CodeQL 대 Joern), 규칙 안전장치, 배포 시 verdict 정책이 다르다. LLMxCPG는 CPG slice 자체를 모델 입력 표현으로 사용하지만, ScanOps의 규칙 생성기는 API 후보 사실을 입력받고 Joern이 실제 taint 경로를 별도로 계산한다. 서로 다른 데이터셋에서 논문 수치와 ScanOps 수치를 직접 순위화하지 않고 구조와 평가 설계만 비교한다.

3. 요구조건 및 설계 변경 사항

착수보고서와 중간보고서의 핵심은 CVEfixes로 파인튜닝한 Qwen3.5-9B 함수 분류기와 tree-sitter/taint 신호를 결합하는 다언어 엔진이었다. 그러나 중간보고서 자체가 기록한 Java test 표본은 69건으로 작았고, PrimeVul·CleanVul과 같은 학습 분포 밖 평가에서는 재현을 저하가 확인되었다. 또한 함수 조각 분류만으로는 “어떤 외부 입력이 어떤 민감 API까지 흘렀는가”를 경로로 설명하기 어려웠다.

최종 단계에서는 분석 대상을 Java로 한정하고, 탐지의 중심을 코드 조각 분류 모델에서 Joern CPG 기반 경로 분석으로 변경했다. Qwen3.8-Max의 역할도 둘로 분리했다. 실험용 규칙 생성 경로에서는 CPG가 추출한 API 후보를 source/sink로 제안하고, 실제 배포 경로에서는 대상 CWE를 알려주지 않은 채 파일 전체를 의미적으로 검토한다. 배포 엔진은 CPG의 구조 경고와 high-confidence 의미 경고를 합치되 Qwen의 무경고가 CPG 경고를 지우지 못하게 한다. 과거 Qwen3.5-9B 파인튜닝 모델과 JS/TS 공용 규칙은 Java 판정에서 제외했고, 비Java 요청에만 기존 RunPod 경로를 유지했다. 즉 CPG 전환의 목적은 단순 성능 경쟁이 아니라 경고 근거의 재현성, 저장소 단위 데이터 흐름, 규칙 출처 통제를 확보하는 데 있다.

표 5. 초기 설계 대비 최종 Java 분석 경로

항목	초기 상태	최종 상태
언어 범위	Java와 JS/TS 자산 혼재	Java만 새 엔진으로 라우팅, 비Java는 기존 경로 유지
LLM	기존 QLoRA와 외부 LLM 혼재	Java는 DashScope Qwen3.8-Max, thinking 비활성
동적 규칙	출처·승격 기준 불명확	shadow 기본, benchmark enforce 별도
분석 중심 평가	함수 조각의 취약/안전 분류 소수 golden/CVE 중심	CPG 경로 분석 + API 역할 제안 Juliet 534 쌍 테스트그룹 + 능력별 분해
지표	F1 중심	P/R/Specificity/FPR/Accuracy/F1/MCC
제품 연결	모델 응답과 백엔드 계약 분리	Java 전용 API·인증·다중 finding·실패 전과 통합
외부 타당성	실제 CVE 자료가 개발 중 소비됨	일반화 우위 미입증, 새 독립 CVE 평가집합 필요

3.1. 평가 집합의 확대

개발 단계의 3개 golden example은 회귀 확인에는 유용하지만 졸업과제의 성능 근거로 부족하다. 이에 정적 분석 도구 시험을 위해 제작된 NIST Juliet Java 1.3[11, 12]에서 현재 Java 규칙·비교 adapter로 실행 가능한 38개 CWE를 먼저 고정했다. 각 CWE에서 취약 동작과 안전 동작이 함께 있는 사례만 남기고 최대 20개 테스트그룹을 선택하여 총 724개를 구성했다. CWE별 상한을 둔 이유는 CWE-89처럼 원본 변형 수가 매우 많은 유형이 전체 점수를 지배하는 것을 막고, 38개 취약 유형을 넓게 비교하기 위해서다. 표본이 20개보다 적은 CWE는 존재하는 쌍을 모두 사용했다.

선택기는 주석과 문자열을 마스킹한 뒤 코드 식별자의 bad/good 표지를 중립 이름으로 바꾼 정규화 hash로 중복을 제거한다. 그 다음 기능 변형 (functional variant)과 흐름 변형 (flow variant)의 현재 선택 횟수가 적은 항목을 우선하고, 동물은 고정 시드 scanops-juliet-economy-v1-20260907와 테스트그룹 ID의 SHA-256으로 결정한다. 이렇게 순서를 먼저 고정한 뒤 각 CWE의 매 네 번째 항목 중 최대 5개를 개발용으로 배정하고 나머지를 평가용으로 두었다. 결과는 개발 190개, 평가 534개이며 최종 표는 평가 534개만 사용한다. 모델이나 탐지 규칙의 결과는 표본 선택과 정답 생성에 사용하지 않았다.

여기서 테스트그룹은 NIST가 제공하는 별도 성능지표 명칭이 아니라 ScanOps 평가기가 Juliet 파일 명명 규칙을 이용해 만든 집계 단위다. 동일 CWE, 기능 변형, 두 자리 흐름 변형 번호가 같은 한 개 이상의 Java 파일을 하나로 묶는다. NIST 설명처럼 Juliet 한 테스트는 한 파일 또는 여러 파일로 구성될 수 있고, bad 부분과 유사 기능의 good 부분을 함께 제공한다[12]. 따라서 파일 수로 세면 다중 클래스 흐름이 과대 가중된다. ScanOps는 각 그룹의 bad 면과 good 면을 각각 한 번씩 판정하여 534쌍을 1,068개의 균형 이진 판정으로 바꿨다.

3.2. 동적 규칙과 고정 규칙의 분리

제품의 동적 규칙 생성 기본값은 shadow다. Qwen이 제안한 API 규칙은 관찰·기록되지만, 명시적인 enforce 없이는 Joern specification에 합쳐지지 않는다. Enforce는 동적 규칙의 실험적 적용이고, promotion은 golden, buggy/fixed, 독립 holdout, 사람 승인 후 java_base.tsv와 registry에 편입하는 별도 절차다. Juliet의 CPG+Qwen 비교군은 eligible exact source/sink를 평가 중에 enforce한 비교 실험이며 운영 고정 규칙으로 승격됐다는 뜻이 아니다.

반면 배포된 cpq-qwen38-ensemble의 파일 의미 검토는 rule shadow와 다른 경로다. 이 분기는 대상 CWE 없이 소스 파일을 Qwen에 보내고 high-confidence 경고만 CPG 결과와 합친다. 따라서 “shadow이므로 배포에서 Qwen이 전혀 쓰이지 않는다”는 해석은 틀리다. 정확히는 API 규칙 제안은 verdict에 반영하지 않고, 별도의 target-blind 의미 경고는 verdict에 반영한다.

3.3. LLM 호출 및 재현성 통제

이 절의 목적은 단순히 API 비용을 줄이는 것이 아니라 입력·설정을 기록하여 비교 조건을 맞추고 실험 변수를 통제하는 것이다. 외부 API의 출력이 매번 같다는 의미는 아니다. Qwen3.8-Max 호출은 enable_thinking=false로 추론 길이 변동을 줄이고, 여러 후보를 한 요청으로 묶는 배치 처리, 동일 후보의 응답 cache, 제한된 재시도를 사용했다. 주 비교 실험에서는 sanitizer, propagation, critic을 끄고 source와 sink 규칙의 효과만 측정했다. 이 설정 덕분에 “Qwen 규칙 생성이 taint 성능에 준 변화”와 “추가 검증 모델이 준 변화”가 섞이지 않는다. 배치·cache는 중복 호출을 줄이기 위한 조치이며, 별도 critic은 추가 토큰 대비 오답 감소가 확인될 때만 채택할 후속 제거 실험 (ablation)으로 남겼다.

3.4. 산학협력 멘토 의견 반영

2026년 8월 11일 한국로봇산업진흥원 김근술 책임의 중간보고서 자문의견은 용어 정의, 채점·벤치마크 과정, AI 개발 단계, 언어별 성능과 원인 분석, 요구조건 추적, 과제 목표의 이유, 상위 Qwen 검토를 보완하도록 요구했다. 최종보고서는 표 6와 같이 대응했다. 중간 단계의 파인튜닝 절차를 실제 최종 엔진에도 존재하는 것처럼 서술하지 않고, QLoRA 분류기를 Java 판정에서 제외하고 Qwen3.8-Max API와 CPG 분석으로 전환한 설계 변경 자체를 명시했다.

표 6. 산학협력 멘토의견과 최종보고서 반영 사항

멘토의견의 핵심	보완·대응 내용	확인 위치
기술 용어 정의 부족	CPG, Joern, source/sink, taint, 테스트그룹과 비교군의 의미를 처음 등장하는 절에서 정의	2-4장
채점 과정과 대상 데이터 부족	Juliet 출처, 38 CWE 선정, 중복 제거, 다양성 순서, 190/534 분할, bad/good 정답과 오류 처리까지 고정	3장, 5장
AI 학습·개발 단계 불명확	중간 QLoRA 파인튜닝 경로와 최종 외부 Qwen+CPG 경로를 구분하고, 최종 Java 엔진은 별도 재학습이 없음을 명시	3장
언어별 평가와 가시화 부족	최종 평가 범위를 Java로 한정하고 1,068판정의 8개 지표와 7/14/17 CWE 구현 능력 층을 표로 분해	5장
벤치마크 신뢰성 부족	NIST Juliet 원본, manifest hash, 동일 split·채점기, 재현 경로와 타당성 위협을 공개	5장, 부록 B
요구조건과 구현의 매칭 부족	초기·최종 설계 비교, 실제 서비스·탐지엔진 아키텍처, 배포 검증 항목을 연결	1장, 3-5장
목표의 이유와 결론 부족	검토 위치와 근거를 제공할 필요성을 설명하고 서비스 구현·성능 평가의 달성 범위를 구분	1장, 8장
상위 Qwen 및 성능 원인 검토	Java 경로를 Qwen3.8-Max로 전환하고, 평균값뿐 아니라 TP/FP 변화와 능력 층별 원인을 분석	3장, 5장

4. 시스템 설계 및 구현

4.1. 사용자가 요청한 코드가 보고서가 되기까지

ScanOps는 사용자가 저장소 주소를 입력하면 코드를 가져와 분석하고, 검토할 위치와 이유를 웹 화면으로 보여주는 시스템이다. 이 절에서는 Java 저장소 요청을 기준으로 설명한다. 웹 URL을 입력하는 동적 분석은 OWASP ZAP을 사용하는 별도 경로이며, 아래 Java 코드 분석과 구분한다.

1. 프론트엔드에서 사용자가 저장소 주소를 제출한다. 백엔드는 스캔 작업을 만들고 분석할 파일을 수집한다.
2. 백엔드가 Java 파일을 Java 전용 분석 서버에 전달한다. 각 파일의 경로와 소스코드를 함께 보내 결과 위치를 원본 파일에 연결한다.
3. 분석 서버는 Joern으로 코드 구조와 데이터 흐름을 검사하고, Qwen에는 파일의 소스코드를 전달하여 보안상 의심되는 동작을 검토하게 한다.
4. 두 분석 결과를 합친 뒤, 분석 상태·취약점 유형·원본 행·판단 근거·탐지 출처를 백엔드에 반환한다.
5. 백엔드는 완료 여부를 검사하여 결과를 저장한다. 프론트엔드는 저장된 보고서를 조회해 사용자에게 보여준다.

Joern은 코드 구조를 만드는 외부 도구이고 Qwen은 외부 언어모델이다. 개발팀이 구현한 부분은 입력 파일을 분석 작업으로 연결하는 코드, 모델 응답을 검사하는 코드, 경고를 합치는 코드, 실패를 처리하는 코드, 결과를 저장하고 보여주는 기능이다. 다음 절에서는 각 부분의 필요성과 동작을 실제 코드로 설명한다. 코드 발췌는 2026년 9월 14일 main에 통합된 구현을 기준으로 하며, 줄 번호는 문서 작성 시 확인한 소스 위치다. 발췌는 처리 부분만 보여주며 전체 함수는 해당 파일에서 확인할 수 있다.

ScanOps 전체 서비스 아키텍처

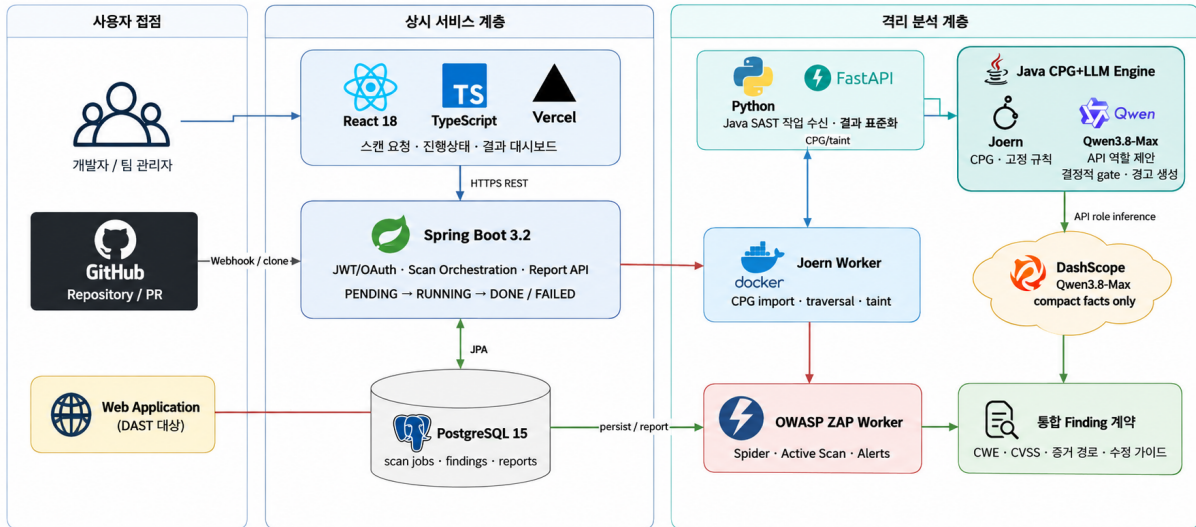


그림 1. 서비스 연결 구조: 사용자 요청부터 분석 서버, 데이터베이스, 결과 화면까지

표 7. 실제 서비스와 두 평가 방식의 차이

구분	처리 흐름	결과의 의미
실제 배포 서비스	Java 파일 → 고정 CPG 분석과 Qwen 파일 검토 → 경고 병합 → 저장·화면 표시	2파일 시연에서 연결 확인. 대규모 탐지 성능은 미측정
규칙 보강 실험	CPG의 API 후보 → Qwen 역할 제안 → 규칙 검사·추가 → CPG 분석·채점	Juliet 534쌍에서 F1 0.400 → 0.466. 오탐도 증가
오프라인 선택 실험	대상 CWE에 따라 분석 결과를 선택·집계	F1 0.901. 대상 CWE를 이용한 실험이며 배포 알고리즘이 아님

배포에서는 Qwen이 파일의 소스코드를 검토하며, 규칙 보강 실험에서는 CPG가 추출한 API 정보와 주변 문맥을 검토한다. 두 경로의 입력과 결과 적용 위치가 다르므로, 규칙 보강 성능을 웹 서비스 전체 성능으로 사용하지 않는다. 아래 4.3~4.4절은 규칙 보강 실험의 구현, 4.2절과 4.5~4.7절은 서비스 분석·결과 처리의 구현을 설명한다.

4.2. Java 파일을 전용 분석 서버로 전달하는 기능

기존 모델 서버 연결만으로는 Java 요청을 새 CPG 분석 서버에 전달할 수 없었으며, Java와 Java Spring Boot 표기가 서로 다른 경로로 처리되면 같은 Java 코드에도 분석 방식이 달라질 수 있었다. 이를 해결하기 위해 백엔드의 ScanopsModelClient.java에서 두 언어 표기를 함께 인식하고, Java 전용 URL이 설정된 경우 해당 서버를 선택하도록 구현하였다. Java 전용 인증키도 기존 비Java 키와 구분하였다. 실제 코드: src/main/java/com/scanops/scan/ScanopsModelClient.java (48~51행)

코드 1. Java 언어 표기와 전용 서버 설정 확인

```
boolean routesToJava(String language) {
    return ("java".equalsIgnoreCase(language)
        || "java spring boot".equalsIgnoreCase(language))
        && javaModelUrl != null && !javaModelUrl.isBlank();
}
```

Java 전용 URL이 있으면 두 표기 모두 새 서버로 전달되고, URL이 비어 있으면 기존 모델 경로를 유지한다. 따라서 배포 시에는 Java URL과 인증키를 함께 설정해야 한다. 언어별 라우팅은 JavaModelRoutingTest.java로 확인하였다. 9월 8일 사이트 연결 기록에는 Java Spring Boot 표기를 기존 모델로 잘못 전달한 실패가 남아 있으며, 이를 수정한 뒤 Java 전용 경로로 시연을 완료했다.

4.3. LLM에 전달할 API 후보와 주변 코드 구성

규칙 보강 실험에서 LLM에 함수 이름만 전달하면 같은 이름의 다른 라이브러리 함수를 구분하기 어렵고, 저장소 전체를 입력하면 어떤 호출에 대한 규칙인지 대응 관계가 흐려질 수 있다. 이를 줄이기 위해 graph_spec_prod.py의 build_items_v2()에서 Joern이 추출한 외부 호출마다 전체 메서드 식별자, 인자·반환형 정보와 최대 세 개의 호출 위치를 묶도록 구성하였다. 아래 코드는 각 위치의 주변 코드와 설명 주석을 추가하는 부분이다. 전체 메서드 식별자(FQN)는 패키지·클래스·메서드와 형식 정보로 호출 대상을 구분하는 값이다.

실제 코드: scanops/core/graph_spec_prod.py (492–495행)

코드 2. API 호출 위치에 주변 코드와 주석 연결

```
for site in (call.get("sites") or []):[:3]:
    path, line = str(site.get("file") or ""), site.get("line")
    sites.append({**site, "context": _source_window(files, path, line),
                  "javadoc": _javadoc_before(files, path, line)})
```

예를 들어 명령 실행 함수 호출이 후보로 들어오면, 출력에는 정확한 호출 대상과 “몇 번째 파일·행에서 어떤 값을 전달했는가”가 함께 기록된다. 내부 메서드의 파라미터는 별도 후보로 만들어 인자 위치를 남기고, 프로젝트 내부 패키지의 일반 호출은 외부 API 후보에서 제외한다.

후보 구성 로직은 test_graph_spec_prod_java.py에서 Runtime.exec 호출 예제를 이용해 검증하였다. 이 테스트는 원본 4행의 코드 문맥과 정확한 메서드 패턴이 유지되는지를 확인하며, 실제 취약점 탐지 성공 여부를 평가하는 테스트와는 구분된다. 이 기능은 규칙 보강 실험에 사용하며, 현재 배포 서비스의 파일 의미 검토와는 입력이 다르다.

4.4. LLM이 제안한 규칙을 검사하여 적용하는 기능

LLM이 “exec라는 이름이면 모두 위험하다”와 같이 범위가 넓은 규칙을 제안하면 관련 없는 API까지 경고할 수 있다. 또한 실제로 확인되지 않은 메서드나 잘못된 인자 위치가 탐지 규칙에 포함되는 문제를 막아야 한다. 이를 위해 후보 ID를 원래 호출 정보와 다시 연결하고, validate()에서 역할·취약점 유형·인자 위치·호출 대상 확인 여부를 검사한다. 입력 지점(source)과 위험 연산 지점(sink)으로 사용할 규칙을 구분하며, 메서드 패턴은 문자열 전체가 일치하도록 생성해 다른 호출로 탐지 범위가 확대되지 않게 하였다.

실제 코드: scanops/core/graph_spec_prod.py (478–479행)

코드 3. 원래 메서드 식별자로 정확한 일치 패턴 생성

```
def _exact_pattern(full_name: str) -> str:
    return "^" + re.escape(full_name) + "$"
```

검증을 통과하고 실험 설정에서 적용이 허용된 후보는 TSV 형식의 규칙으로 변환되어 Joern에 전달된다. Joern은 입력 지점부터 위험 연산까지의 데이터 흐름이나 직접 규칙 조건을 검사하므로, LLM이 위험 함수라고 제안했다는 사실만으로 취약점을 확정하지 않는다.

예를 들어 SQL 실행 API가 기존 위험 함수 목록에서 누락되었다면, 이 기능은 해당 API를 정확한 메서드 식별자로 sink 규칙에 추가한다. 그 뒤 Joern이 외부 입력부터 해당 호출까지의 경로를 확인한다. LLM이 API를 위험하다고 제안하는 것만으로 데이터 흐름이나 취약점이 확정되지는 않는다. 이는 처리 원리를 설명하는 예이며 새로 관측한 탐지 사례는 아니다.

규칙 테스트에서는 정확한 호출·높은 신뢰도·유효한 인자 위치를 가진 후보만 적용되는지, 호출 대상을 확인하지 못했거나 범위가 넓은 패턴은 제외되는지를 확인하였다. 최종 비교 실험에서는 허용된 source/sink 규칙을 임시로 적용했다. 현재 배포에서는 규칙 생성을 끄고 추가 규칙을 판정에 적용하지 않는 shadow 설정을 사용하며, 이와 별도로 다음 절의 Qwen 파일 검토가 실행된다.

4.5. 긴 파일을 나누어 검토하고 원본 행을 복원하는 기능

파일 앞부분만 잘라 모델에 전달하면 뒤쪽 코드를 검사할 수 없고, 파일을 여러 조각으로 나누면 모델이 반환한 행 번호가 원본 파일의 위치와 달라질 수 있다. 이를 처리하기 위해 java_semantic.py의 review()

는 기본 24,000자 단위로 파일을 나누되 최대 1,000자가 겹치도록 하고, 각 조각 앞에 있는 줄 수를 더해 모델이 반환한 행 번호를 원본 위치로 복원한다. 다음은 해당 처리 부분이다.

실제 코드: scanops/core/java_semantic.py (76–83행)

코드 4. 파일 조각의 검토 결과를 원본 행으로 복원

```
result = _review_window(code[offset:end], call, max_chars)
base_line = code.count("\n", 0, offset)
windows.append({"start_offset": offset, "end_offset": end,
               "status": result["status"]})
completed += result["status"] == "DONE"
for finding in result["findings"]:
    f = {**finding, "line": finding["line"] + base_line,
         "context_scope": "overlapping-window"}
```

예를 들어 조각 앞에 100줄이 있고 모델이 조각의 5행을 반환하면 원본 105행으로 표시한다. 이는 행 번호 복원 원리를 설명하기 위한 예다. 모든 조각을 처리하더라도 서로 다른 조각이나 파일 사이의 관계까지 모델이 정확히 추론한다고 보장할 수는 없으며, 조각 하나라도 실패하면 전체 파일을 정상 완료로 처리하지 않는다.

test_java_semantic.py에서는 입력 끝까지 조각이 생성되는지와 여러 호출 중 하나가 실패할 때 PARTIAL 상태가 반환되는지를 확인한다. 이 테스트는 모의 응답을 사용하므로 외부 모델의 실제 탐지 정확도 평가와는 구분된다.

4.6. 두 분석기의 경고를 합치고 출처를 보존하는 기능

CPG와 Qwen이 같은 파일의 같은 취약점을 각각 반환하면 중복 경고가 생길 수 있다. 반대로 Qwen이 경고하지 않았다는 이유로 CPG 경고를 제거하면 기존 구조 분석 결과가 사라진다. merge()는 한 파일 안에서 CWE와 원본 행이 같은 경고를 하나로 합치고, 어떤 분석기가 경고했는지는 contributors에 남긴다. CPG 경고는 모두 유지하며, Qwen 경고는 응답 검사를 통과한 high-confidence 후보만 추가한다. 여기서 high는 모델이 반환한 신뢰도 표기이며 정확도가 보장된 확률을 의미하지 않는다.

실제 코드: scanops/core/java_semantic.py (105–114행)

코드 5. 경고 병합: 중복은 합치고 탐지 출처는 유지

```
merged: dict[tuple, dict] = {}
for finding in cpg_findings + [f for f in semantic_findings if f["accepted"]]:
    key = (finding.get("cwe"), finding.get("line"))
    source = finding.get("source", "cpg")
    if key in merged:
        if source not in merged[key]["contributors"]:
            merged[key]["contributors"].append(source)
    else:
        merged[key] = {**finding, "contributors": [source]}
return list(merged.values())
```

표 8. 경고 병합의 동작 예시: 실제 탐지 성능과 별도의 로직 설명

CPG 입력	Qwen 입력	병합 결과
CWE-798, 1행	같은 CWE·행, high	경고 1건, 두 출처 기록
CWE-798, 1행	경고 없음	CPG 경고 유지
경고 없음	CWE-89, 1행, low	최종 경고에 추가하지 않음
CWE-78, 1행	CWE-798, 1행, high	유형이 달라 경고 2건 유지

기존 의미 검토 테스트에서는 중복 경고 1건에 두 출처가 함께 남는지, 낮은 신뢰도 경고가 추가되지 않는지, 서로 다른 두 경고가 API 결과에 각각 보존되는지를 확인하였다. 위 표는 이러한 병합 규칙을 설명하기 위한 예시이며 새로운 실제 취약점 발견 결과는 아니다.

4.7. 분석 실패를 구분하고 결과를 저장·표시하는 기능

분석 서버가 응답하지 않거나 요청 파일 중 일부를 처리하지 못했을 때 빈 경고 목록만으로 “안전하다”고 표시하면 분석 실패를 정상 결과로 오인할 수 있다. 또한 복수 경고를 파일당 한 건으로 축약하면 실제 취약 위치가 누락될 수 있다. 이를 방지하기 위해 백엔드에서 응답의 파일 경로와 완료 상태를 검사하고, 조건을 통과하지 못한 응답은 정상 결과로 저장하지 않고 오류로 전달하도록 하였다. 배치 경로에서는 파일 수와 대응 관계도 함께 확인한다.

실제 코드: src/main/java/com/scanops/scan/ScanopsModelClient.java (62–66행)

코드 6. 응답 파일·완료 상태 검사

```
private void validate(AnalyzeResult r, AnalyzeRequest request, boolean javaRoute) {
    if (r == null || !Objects.equals(r.file_path(), request.file_path())
        || (javaRoute && !"DONE".equals(r.status()))
        || (r.status() != null && !"DONE".equals(r.status())))
        throw new IllegalStateException("Analysis incomplete or mismatched");
}
```

AnalyzeResult.individualResults()는 한 파일의 여러 경고를 개별 결과로 풀어 원본 위치와 출처를 보존한다. GithubPipelineRunner.java가 이를 처리하여 취약점과 작업 상태를 저장한다. 프론트의 reportApi.ts는 /api/reports/{id}를 조회하고 ReportPage.tsx가 취약점 설명과 수정 안내를 표시한다.

2026년 9월 8일 사이트에서는 Java 2파일·25행을 대상으로 연결 시연을 수행했다. 요청부터 결과 저장·표시까지 약 37초가 걸렸고, VulnerableServlet.java의 원본 12행에 CWE-79 경고 1건이 표시됐다. 출처는 CPG였으며 SafeServlet.java에는 경고가 없었다. 이를 통해 요청·분석·저장·화면 표시가 연결되어 동작하는 것을 확인했다. 다만 사이트에서 Qwen 단독 경고와 복수 경고 UI가 표시되는 경우까지 확인한 것은 아니다.

4.8. 직접 구현한 기능과 확인 근거 정리

표 9. 구현 기능에서 검증 자료까지의 연결

직접 구현한 기능	확인할 코드·자료	확인한 범위
Java 전용 라우팅	ModelClient, JavaModelRoutingTest	언어·설정별 호출 경로
후보·문맥 구성	build_items_v2, 후보 구성 테스트	호출 식별자·주변 행 연결
제안 규칙 검사	validate, 규칙 적용 테스트	허용·제외 조건
파일 분할·경고 병합	review, merge, 의미 검토 테스트	입력 처리·중복·오류 동작
저장·화면 연결	배포 기록, 사이트 검증 기록	2파일 시연에서 CPG 경고 표시
규칙 보강 효과 비교	Juliet 확정 채점 결과	평가 534쌍의 탐지·오탐 변화

테스트 통과는 작성한 처리 규칙이 의도대로 동작하는지에 대한 근거다. 탐지 성능은 다음 장의 정답 데이터 비교로, 서비스 연결은 사이트 시연 기록으로 각각 확인한다. 전체 주요 파일 목록과 읽는 순서는 부록 D에 정리하였다.

5. 실험 방법, 결과 및 논의

먼저 확인한 것은 “추가 규칙이 기존에 놓친 취약 코드를 더 찾는가, 그 대신 안전한 코드에서 경고가 얼마나 늘어나는가”이다. 따라서 취약 코드만이 아니라 유사한 안전 코드도 함께 평가하였다. 다음 표의 성능 수치는 규칙 보강 실험의 결과이고, 실제 사이트 연결 성공은 별도 항목에서 다룬다.

이 장에서는 서로 다른 세 결과를 분리한다. 첫째는 Juliet에서 동일 표본과 채점 기준으로 네 비교군

을 측정한 통제 실험, 둘째는 그 결과로 구성한 대상 **CWE** 인지형 오프라인 **router** 후보, 셋째는 실제 사이트에 배포한 대상 **CWE** 비인지형 **CPG-Qwen union** 엔진이다. 둘째와 셋째는 결합 방식과 입력이 다르므로 F1 0.901을 배포 엔진의 성능으로 사용하지 않는다.

5.1. 데이터 출처, 선택 및 분할

원본은 NIST SARD의 Juliet Java 1.3이다. 이 모음은 정적 분석 도구 시험을 위해 만든 인공 코드 28,881건과 112개 CWE를 포함하며, 각 테스트는 원칙적으로 한 취약 유형을 목표로 하고 유사 동작의 bad와 good 구현을 제공한다[11, 12]. ScanOps는 전체 112개를 지원한다고 주장하지 않는다. 현재 Java rule catalogue와 네 비교 adapter가 모두 결과를 낼 수 있는 38개 CWE를 평가 범위로 고정했다.

표 10. Juliet 평가집합 생성 근거

항목	고정 규칙과 이유
원본 포함 조건	NIST Juliet Java 1.3; 원본 파일 hash를 manifest에 저장 파일명에서 CWE·기능 변형·흐름 변형을 해석할 수 있고 bad와 good이 모두 존재
집계 단위	같은 CWE, 기능 변형, 흐름 변형 번호의 1개 이상 파일을 테스트그룹 1개로 묶음
중복 통제	주석 제거·bad/good 식별자 중립화 후 정규화 SHA-256이 같은 그룹은 하나만 유지
다양성·상한 동물 결정	기능 변형과 흐름 변형의 선택 횟수가 적은 순으로 CWE당 최대 20개 고정 시드와 테스트그룹 ID의 SHA-256 순; 실행할 때마다 동일
분할	고정된 순서의 매 4번째 그룹 중 최대 5개/CWE는 개발, 나머지는 평가
최종 수량	38 CWE, 전체 724쌍 = 개발 190쌍 + 최종 평가 534쌍
독립성	표본 선정과 정답 생성에 ScanOps·Qwen·CodeQL 출력 미사용
무결성	평가 manifest 내용 SHA-256: 74870c35...e5bf

38개 중 26개 CWE는 20쌍을 확보해 개발 5, 평가 15로 나뉘었다. 원본에 17쌍뿐인 12개 CWE는 개발 5, 평가 12가 되었다. 따라서 최종 평가는 $26 \times 15 + 12 \times 12 = 534$ 쌍이다. 정확한 CWE별 ID와 수량은 확정된 manifest를 기준으로 하며, 각 쌍의 bad와 good을 독립 판정하므로 판정 수는 $534 \times 2 = 1,068$ 이다. 이 균형은 accuracy가 다수 클래스만 맞춰 높아지는 현상을 줄이지만 실제 취약 코드의 발생률을 재현하지는 않는다.

Source-only 입력에서는 한 쌍의 bad와 good을 동시에 보여주지 않았다. 반대쪽 메서드를 제거하고 공용 helper만 남긴 뒤 주석을 제거하고 bad/good이 들어간 식별자를 variant_N으로 바꿨다. 누출 검사에서 flaw, potential flaw, fix, bad/good 식별자 출현은 모두 0이었다. 각 면은 무작위화된 독립 요청이며, CPG 비교군은 같은 manifest 파일들을 저장소 단위로 Joern에 import했다.

5.2. 비교군 선택과 문헌 근거

IRIS는 CodeQL baseline과 LLM으로 보강한 정적 분석을 같은 Java CVE 집합에서 비교하고, source/sink 사양 및 contextual filtering의 기여를 분리했다[5]. LLMDEFA는 Juliet에서 precision, recall, F1뿐 아니라 source/sink 추출과 data-flow 단계별 성능을 측정했다[7]. GPTScan과 RepoAudit도 각각 프로그램 분석 결합 전후 및 validator/cache 제거 실험으로 구성요소 기여를 확인했다[8, 10]. 이에 따라 ScanOps의 주 비교는 “어떤 상용 챗봇이 더 좋은가”가 아니라 “같은 Qwen에서 구조 분석과 규칙 보강이 무엇을 바꾸는가”로 설계했다.

GPT·Claude를 추가하면 모델 능력과 아키텍처 효과가 동시에 바뀌어 평가 3의 인과 해석이 약해진다. 따라서 Qwen3.8-Max를 고정하고 LLM 유무·입력 표현·정적 분석기를 바꿨다. 이는 여러 모델을 비교한 IRIS의 별도 평가 항목을 부정하는 것이 아니라 이번 졸업과제의 통제 변수를 줄인 선택이다. 결과적으로 “Qwen보다 우수하다” 또는 “GPT·Claude보다 우수하다”는 주장은 하지 않는다.

5.3. 정답, 출력 매핑 및 지표

Juliet의 파일명과 bad/good 메서드 관례만 정답으로 사용했다. 한 테스트그룹의 bad 면은 양성 1건, good 면은 음성 1건이다. 탐지기가 해당 면의 메서드 구간 또는 그 경로 노드에서 그 그룹의 목표 **CWE**와

표 11. 동일 manifest의 네 주 비교군

비교군	실제 입력과 출력	비교 목적과 제한
고정 CPG	Java 전체 파일 → Joern + java_base.tsv/sanitizer → 경로 finding	LLM 없는 ScanOps 구조 기준선
CPG+Qwen 규칙	CPG API 후보 사실 → exact source/sink 제안 → Joern taint finding	규칙 보강 효과; 실험에서만 enforce, 배포 승격 아님
Qwen source-only	한 테스트 먼의 중립화된 소스 + 대상 CWE → 취약/안전 JSON	의미 모델 기준선; target-CWE-aware closed set
CodeQL	같은 파일 → CodeQL 2.23.2, Java query pack 1.8.1 → alert	외부 정적 분석 기준선; 선택 query/adapter 범위

표 12. 관련 하이브리드 기법과 평가 설계의 연결

문헌	구조	평가 관점	ScanOps에 반영한 점
IRIS[5]	CodeQL + LLM 사양·경로 필터	CodeQL, 8개 LLM, 구성요소 분석	고정 분석 대 LLM 사양 보강, 동일 CWE 채점
LLMDFA[7]	LLM data-flow + parser/SMT	Juliet P/R/F1, 단계별 평가	전체와 source/sink-taint 층을 분리
GPTScan[8]	LLM 의미 이해 + 정적 분석	도구 비교와 구성요소 효과	LLM 단독과 프로그램 분석 결합 분리
LLMxCPG[9]	CPG slice + LLM	함수·다중 함수, 강건성	전체 그래프 대신 제한된 구조 문맥 사용
RepoAudit[10]	경로 탐색 agent + validator	산업 도구, ablation, 비용	실패·cache·검증 정책까지 결과와 분리 기록

동일한 **finding**을 하나 이상 출력할 때만 양성 예측으로 인정한다. 다른 CWE 경고는 off-target으로 별도 집계하고 정답으로 바꾸지 않는다. 한 경로가 bad와 good 양쪽에 걸리면 ambiguous로 기록하며 TP로 자동 전환하지 않는다. 파일·행에 매핑되지 않는 경고도 unmapped로 남긴다.

따라서 bad를 맞히면 TP, 놓치면 FN이고 good을 잘못 올리면 FP, 올리지 않으면 TN이다. 네 비교군은 동일 manifest와 채점기를 쓴다. Qwen source-only에서 반복된 파싱 실패 2건도 평가에서 제외하지 않고 FN으로 처리했다.

표 13. 성능 지표의 계산과 해석

지표	계산	본 평가에서 답하는 질문
Precision	$TP/(TP + FP)$	발생한 경고 중 목표 취약점인 비율
Recall	$TP/(TP + FN)$	목표 취약 bad 구현을 찾은 비율
Specificity	$TN/(TN + FP)$	good 구현을 안전으로 구분한 비율
FPR	$FP/(FP + TN)$	good 구현을 잘못 올린 비율
Accuracy	$(TP + TN)/(TP + FP + FN + TN)$	1,068개 전체 판정의 정답 비율
F1	$2PR/(P + R)$	경고 정확성과 미탐 사이의 조화평균
Macro F1	CWE별 F1의 산술평균	표본 수와 무관하게 CWE를 같은 가중치로 본 값
MCC	네 혼동행렬 항의 상관관계 수	양·음성 예측의 전체 상관; -1에서 1

F1은 precision이나 recall 한 지표만으로 성능을 해석하는 한계를 줄이기 위해 대표값으로 사용했지만, TN을 직접 반영하지 않는다. 따라서 specificity, FPR, accuracy, MCC와 혼동행렬을 항상 함께 제시한다. 균형 표본이므로 본 평가의 accuracy와 balanced accuracy는 수치상 같다.

표 14. Juliet 최종 평가 38 CWE, 534쌍(1,068판정)

Arm	TP	FP	FN	TN	P	R	Spec.	FPR	Acc.	F1	Macro F1	MCC
고정 CPG	144	42	390	492	.774	.270	.921	.079	.596	.400	.270	.252
CPG+Qwen 규칙	183	69	351	465	.726	.343	.871	.129	.607	.466	.336	.251
Qwen source-only	508	107	26	427	.826	.951	.800	.200	.875	.884	.885	.760
CodeQL	115	40	419	494	.742	.215	.925	.075	.570	.334	.201	.199

5.4. 전체 38 CWE 결과

CPG+Qwen 규칙은 고정 CPG보다 TP가 39 증가하고 FN이 39 감소하여 recall이 0.270에서 0.343, F1이 0.400에서 0.466으로 증가했다. 동시에 FP가 42에서 69로 늘어 FPR이 0.079에서 0.129가 되었고 precision은 0.774에서 0.726으로 감소했다. MCC는 0.252에서 0.251로 사실상 같으므로 “전반적 품질 향상”보다 탐지 범위를 넓혔지만 오탐 비용도 늘었다고 해석해야 한다.

Qwen source-only는 F1 0.884로 가장 높았지만 대상 CWE를 입력받은 closed-set 조건이고 FPR도 0.200이다. 입력에서 주석과 답 표지를 제거했어도 Juliet의 합성·반복 구조와 목표 CWE 자체가 강한 힌트가 된다. 실제 제품처럼 CWE를 모르는 저장소 전체를 한 번 검토하는 조건과 동일하지 않으므로 배포 성능으로 일반화하지 않는다. CodeQL 수치도 사용한 버전, query pack, Juliet adapter에 한정하며 CodeQL 제품 전체의 절대 성능으로 해석하지 않는다.

5.5. 고정 규칙의 구현 방식별 결과

전체 평균의 원인을 보기 위해 java_base.tsv의 matcher 종류만으로 세 층을 만들었다. exists, exists_full, arg_literal, arg_count가 있는 7개 CWE를 “직접 API·인자”, source/sink의 name/full/code가 있는 14개를 “경로 기반 taint”, 목표 CWE 규칙이 없는 17개를 “고정 규칙 미구현”으로 분류했다. 이 층화는 완전한 비교 점수가 나오기 전에 고정했으며, CPG가 이론적으로 표현 가능한지에 대한 분류가 아니라 현재 구현에 어떤 **matcher**가 존재하는가에 대한 설명 변수다.

표 15. 직접 API·인자 규칙 7 CWE, 102쌍

Arm	TP	FP	FN	TN	Precision	Recall	FPR	F1
고정 CPG	102	0	0	102	1.000	1.000	.000	1.000
CPG+Qwen 규칙	102	0	0	102	1.000	1.000	.000	1.000
Qwen source-only	86	2	16	100	.977	.843	.020	.905
CodeQL	35	0	67	102	1.000	.343	.000	.511

CWE-114, 209, 327, 328, 338, 382, 477의 204판정에서 고정 CPG는 FP와 FN 없이 F1 1.000이었다. 이 결과는 전체 38 CWE 성능이 아니라 exact API 존재, literal, 인자 수로 이미 구현한 좁은 구간의 결과다. 이 구간에서는 Qwen 규칙을 더해도 이득이 없으므로 제품의 의미 검토를 생략할 수 있는 비용 절감 후보가 된다.

표 16. Source-to-sink taint 14 CWE, 210쌍

Arm	TP	FP	FN	TN	Precision	Recall	FPR	F1
고정 CPG	42	42	168	168	.500	.200	.200	.286
CPG+Qwen 규칙	73	53	137	157	.579	.348	.252	.435
Qwen source-only	202	35	8	175	.852	.962	.167	.904
CodeQL	65	25	145	185	.722	.310	.119	.433

CWE-15, 23, 36, 78, 80, 81, 83, 89, 90, 113, 134, 470, 601, 643에서는 Qwen exact API 규칙이 고정 CPG 대비 recall을 20.0%에서 34.8%로 1.74배, F1을 0.286에서 0.435로 높였다. TP는 42에서 73으로 31건 늘었으며, 이것이 CPG와 API 의미 제안을 결합할 실험적 근거다. 그러나 FP도 42에서 53으로 늘어 FPR이 5.2%p 악화되었다. 따라서 “Qwen이 CPG를 항상 개선한다”가 아니라 누락 **source/sink**를 보완해 **recall**을 높이되 승격 검증과 **sanitizer**가 필요하다는 결론이다.

표 17. 현재 고정 규칙 미구현 17 CWE, 222쌍

Arm	TP	FP	FN	TN	Precision	Recall	FPR	F1
고정 CPG	0	0	222	222	.000	.000	.000	.000
CPG+Qwen 규칙	8	16	214	206	.333	.036	.072	.065
Qwen source-only	220	70	2	152	.759	.991	.315	.859
CodeQL	15	15	207	207	.500	.068	.068	.119

이 17개는 hardcoded secret, 정보 노출, 임시 파일, 설정 의미처럼 현재 fixed spec에 목표 matcher가 없는 유형이다. CPG로 원천적으로 불가능하다는 뜻은 아니며 literal, initializer, configuration, resource lifecycle 규칙을 추가해야 한다. Qwen 규칙 생성도 API role 중심이라 이 구간에서 recall 0.036에 그쳤다. 반면 target-aware source-only는 잘 분류했으나 FPR 0.315로 의미 단독 경고의 위험도 함께 보였다.

5.6. 오프라인 능력 기반 router 후보

개발 결과와 fixed matcher 능력을 기준으로 직접 7 CWE는 고정 CPG, 나머지 31 CWE는 source-only를 선택하는 route를 완전한 평가 결과가 나오기 전에 파일로 고정했다. 평가 534쌍에 적용하면 TP 524, FP 105, FN 10, TN 429, precision 0.833, recall 0.981, FPR 0.197, F1 0.901, MCC 0.797이었다. Qwen 호출 대상도 전체 38개 중 31개로 줄어 평가 단위 기준 약 19.1% 감소한다.

추가 검토 대상의 정확한 의미

F1 0.901은 “CPG가 완전 판정한 7개 target CWE는 CPG, 나머지는 target-CWE-aware Qwen”으로 선택한 오프라인 결과다. 실제 저장소는 정답 CWE를 모르므로 이 수치는 배포 엔진 성능이 아니며 제품 우위 주장에 사용하지 않는다.

이 후보는 역할 분리 가능성을 보여주지만, 배포에 그대로 붙이면 정답 CWE를 routing key로 쓰는 label leakage가 된다. 제품 router는 CPG import 성공, matcher 지원 여부, 경로 완전성, evidence level 같은 관측 가능한 값만으로 선택해야 하고 별도 blind 평가를 통과해야 한다.

5.7. 실제 배포 엔진과 통합 검증

실제 배포 모드는 cpg-qwen38-ensemble이고 정책은 java-semantic-union-v2-full-file-windows다. Joern의 고정 CPG finding을 보존하고, 대상 CWE 없이 파일을 검토한 Qwen 결과 중 high-confidence만 추가한다. 같은 CWE·행은 contributor를 합치고, Qwen 무경고는 CPG 경고를 취소하지 않는다. 규칙 생성은 배포에서 비활성·shadow이므로 Juliet의 CPG+Qwen 규칙 비교군과도 다르다.

표 18. 2026년 9월 8일 시험 배포 검증

검증 항목	관찰 결과
구성	별도 t3.large 분석 EC2, FastAPI + Joern Docker, 외부 DashScope
라우팅	Java/Java Spring Boot만 새 엔진; 비Java는 기존 RunPod 유지
인증·실패	올바른 키 200, 잘못된 키 401; PARTIAL·파일 누락은 실패 처리
모델·백엔드 시험	모델 58 tests + 5 subtests, 백엔드 37 tests, 실패 0
취약 smoke	CaseA.java CWE-79 8행; CPG와 Qwen-semantic contributor 일치
안전 smoke	SafeCase.java 경고 0건
실제 사이트	2개 Java 파일, 25행, 약 37초, 작업 완료 및 DB·화면 표시
사이트 finding	VulnerableServlet.java 12행 CWE-79 1건, source=CPG
사이트 안전 파일	SafeServlet.java 경고 없음
미검증	대형 저장소 부하, 복수 경고 UI, Qwen 단독 사이트 finding, 일반화 성능

모델 API smoke는 CPG와 Qwen 기여 경고의 병합을 확인했고, 사이트 시연은 Java 요청부터 CPG 경고의 DB 저장·화면 표시까지 연결됨을 확인했다. Qwen 단독 경고가 사이트에 표시되는 경우는 이 시연에서 관측하지 못했다. 그러나 두 파일은 의도적으로 만든 개발 시연 자료이므로 precision, recall, F1 또는 실제 CVE 탐지 성능의 근거가 아니다. 특히 배포 union은 CPG의 오탐을 Qwen이 제거하는 critic

구조가 아니어서, 의미 모델을 추가하면 recall은 늘 수 있지만 FP가 유지되거나 늘 수 있다.

5.8. 실제 CVE 탐색 평가와 보고 범위

별도의 구조 규칙 후보 pack 검증은 배포된 target-blind union과 입력·결합 정책이 달라 제품 성능 근거에서 제외했다. 이후 실제 Java CVE에 target CWE와 fix 위치를 주지 않는 유료 blind 평가를 시작했으나 사용자가 중단할 때까지 8개 중 첫 프로젝트 26/26 단계만 완료되고, 둘째는 63/63 중 semantic 실패 1건으로 PARTIAL, 셋째는 규칙 생성 도중 중단, 나머지 5개는 미실행이었다. 총 201회 호출의 부분 산출물은 서로 다른 완료 상태이므로 하나의 혼동행렬로 합치지 않았다.

추가 repository-context 실험은 Jenkins의 취약·수정 쌍 두 개에 대해 네 응답을 얻었으나 모두 CWE-668 medium confidence였고, 수정본에서도 근거가 부족한 위험을 추측했다. high-confidence로 수용된 finding은 0건이며 buggy/fixed 구분 개선을 입증하지 못했다. 따라서 본 보고서의 정량 주장은 Juliet 통제 실험에 한정하고, 실제 CVE에 대해서는 평가를 시도했으나 불완전했고 일반화 성능을 입증하지 못했다고 결론낸다.

5.9. 재현성, 실패 처리와 타당성 위험

모든 최종 수치는 manifest 내용 hash가 같은 산출물만 결합했다. Joern 병렬 비교군의 임시 job ID 충돌은 arm 이름을 포함하도록 수정하고 유효 checkpoint에서 재개했다. import 실패가 남은 두 CWE는 해당 CWE만 격리 재실행해 38개가 모두 성공한 뒤 채점했다. Off-target finding은 고정 CPG 137건, CPG+Qwen 219건이었으나 목표 CWE 정답으로 세지 않았다. CPG+Qwen의 ambiguous target finding 2건도 자동 TP로 바꾸지 않았다.

주요 타당성 위험은 네 가지다. 첫째, Juliet은 합성·균형 데이터라 실제 발생률과 프레임워크 다양성을 반영하지 않는다. 둘째, source-only는 target CWE를 입력받아 open-world 배포보다 쉽다. 셋째, 능력 증은 이론적 CPG 가능성이 아니라 현재 규칙 구현 상태다. 넷째, 배포 union은 대규모 정답 데이터로 채점되지 않았다. 그러므로 최종 결론은 “Juliet의 현재 38 CWE 범위에서 어떤 구성요소가 어떤 지표를 바꿨는가” 까지이며, 실제 CVE 순위는 후속 독립 평가 과제다.

5.10. 평가 결과 요약과 활용 시 고려사항

평가 1의 전체 비교에서 Qwen 단독은 F1 0.884로 가장 높았고, 규칙 보강 CPG는 0.466이었다. 다만 단독 비교군에 대상 CWE를 제공했으므로 동일 표본을 썼다는 사실만으로 입력 정보까지 동일하다고 볼 수 없다. 평가 2에서는 직접 API·인자 규칙 7종과 데이터 흐름 추적 규칙 14종, 고정 규칙 미구현 17종의 차이가 전체 평균을 설명했다. 이는 CPG 이론의 한계 분류가 아니라 현재 구현 범위에 대한 결과다.

평가 3에서는 규칙 보강으로 TP가 39건 늘었으나 FP도 27건 늘었다. 그러므로 추가 규칙을 운영에 채택할 때는 새로 찾는 취약점뿐 아니라 안전 코드에서 늘어나는 경고를 함께 검토해야 한다. 평가 4에서는 규칙 보강 평가, 대상 CWE 기반 선택 실험, 배포 합집합 방식의 입력과 정책이 달라 결과를 서로 대체할 수 없음을 확인했다. 이번 비교를 통해 어떤 규칙을 보완했을 때 탐지가 늘고 어떤 오탐 부담이 생기는지를 확인하였다.

6. 구성원별 역할 및 개발 일정

6.1. 구성원별 역할 및 최종 진척

착수·중간보고서와 저장소별 구현 범위를 바탕으로 프론트엔드, 탐지 엔진, 백엔드·인프라 역할을 다음과 같이 정리하였다. 탐지 엔진은 김세환과 이경윤이 공동 개발하되, 제품 UI·성능평가와 Joern 분석 기반을 나누어 담당했다.

표 19. 구성원별 역할 및 최종 진척

구성원	작업 영역	세부 진척
김세한	프론트엔드·CPG/LLM 탐지 엔진	React/TypeScript 화면과 스캔 UX, Java 경로 분리, Qwen3.8-Max 규칙 생성·검증 gate, Juliet 비교평가 및 결과 해석
이경윤	CPG/LLM 탐지 엔진	Joern 실행환경과 CPG/taint 질의, 데이터 흐름 및 후보 추출 실험, 엔진 재현·안정화 공동 수행
전혜은	백엔드·인프라	Spring Boot API, 인증·DB·스캔 오케스트레이션, 분석 서버 연동, AWS/Docker 및 OWASP ZAP 운영 경로

6.2. 개발 일정

표 20은 전체 프로젝트 착수일을 임의로 바꾼 것이 아니라, 중간보고 이후 **Java CPG+LLM** 최종 고도화가 집중된 **8-9월** 일정만 월 단위로 재구성한 것이다. 중간보고 단계의 파인튜닝 모델·프론트엔드·백엔드 작업은 선행 결과로 취급한다.

표 20. 2026년 8-9월 Java CPG+LLM 집중 개발 일정

단계	시점	산출물
기존 구조 진단·Joern 통합	2026.08	파인튜닝 모델 한계 분석, CPG worker, taint query
규칙 생성 고도화	2026.08	API 후보 추출, 외부 API 역할 제안, cache·문맥 처리
Java 전용 경로 분리	2026.09	exact FQN 후보, source kind, shadow/enforce gate
실제 CVE 승격 검증	2026.09	취약/수정 쌍 실패 분석과 후보 승격 거부
Juliet 대규모 평가	2026.09	38 CWE, 534 쌍 테스트그룹, 4개 비교군
Java 시험 배포·사이트 통합	2026.09	분석 EC2, Java 전용 라우팅, DB·화면 end-to-end 검증
최종보고·포스터	2026.09	부산대 졸업과제 제출 형식 문서와 재현 가이드
독립 CVE 평가·배포 양 상블 채점	후속 검증	새 실제 CVE와 target-blind union/선택 정책 비교

7. 현실적 제약 사항 및 대책

- **LLM** 호출량: Qwen3.8-Max의 thinking을 비활성화하고 batch와 cache를 사용했다. 배포 의미 검토는 파일 단위로 수행하되 긴 파일은 여러 창으로 나누며 CPG 확정 구간의 생략은 아직 추가 검토 대상이다.
- **LLM** 비결정성: Temperature를 낮추고 JSON schema, exact FQN 재부착, high-confidence gate, 제한된 재시도를 사용한다.
- **CPG import** 안정성: Arm별 고유 임시 작업 ID와 checkpoint validation, 실패 CWE 격리 재실행을 적용한다.
- 고정 규칙의 범위: FN을 API rule, import, path coverage, sanitizer, propagation으로 분해해 개선한다.
- 평가 일반화: Juliet은 합성 데이터이며 source-only는 target-CWE-aware다. 중단된 실제 CVE 부분 실행은 점수로 사용하지 않고 새 독립 집합을 사전 고정한다.
- 동적 규칙 안전성: 제품의 API 규칙 생성은 비활성·shadow를 유지하고 buggy/fixed·holdout·사람 승인 없이 fixed spec으로 승격하지 않는다. 의미 union은 출처를 별도로 표시한다.
- 비교 도구 해석: 본 CodeQL 수치는 선택한 query pack과 Juliet adapter의 결과이며 CodeQL 전체 제품 성능으로 일반화하지 않는다.

8. 결론 및 향후 개선 계획

본 과제에서는 Java 코드를 분석하고 검토할 위치와 이유를 웹 보고서로 보여주는 ScanOps를 구현했다. 개발팀은 Java 전용 서버 연결, API 후보와 주변 코드 구성, LLM 응답 검사, 긴 파일의 분할과 원본 행 복원, 중복 경고 병합, 분석 실패 처리, 결과 저장과 화면 표시를 담당하였다. Joern과 Qwen은 외부 도구로 활용했고, 두 도구의 출력을 사용자가 확인할 수 있는 하나의 결과로 연결하였다.

통제 실험은 NIST Juliet Java의 38개 CWE에서 중복과 답 표지를 제거하고 다양성 기준으로 고정한 534개 취약·안전 쌍, 1,068개 판정이다. 전체에서 CPG+Qwen 규칙의 F1은 0.466으로 고정 CPG 0.400과 선택한 CodeQL 기준선 0.334보다 높았지만 target-CWE-aware Qwen source-only 0.884보다 낮았다. 직접 API·인자 7 CWE의 고정 CPG는 102쌍에서 F1 1.000, FPR 0이었다. Taint 14 CWE에서는 Qwen 규칙 보강이 recall을 0.200에서 0.348로 1.74배, F1을 0.286에서 0.435로 높였으나 FPR도 0.200에서 0.252로 증가했다.

대상 CWE를 이용하는 오프라인 router 후보는 F1 0.901을 기록했지만 실제 배포 알고리즘이 아니다. 배포된 target-blind union은 개발 시연에서 end-to-end 동작을 확인했으나 대규모 정답 집합으로 성능을 측정하지 못했다. 실제 CVE 유료 평가는 일부 프로젝트만 처리한 뒤 중단되어 혼동행렬을 만들 수 없었고, 후속 context 실험도 buggy/fixed 구분을 입증하지 못했다. 따라서 최종 확인 결과는 Juliet의 통제 조건에서 CPG가 강한 구간과 Qwen 규칙의 recall 보완 효과를 확인했고 제품 통합에는 성공했지만, 실제 CVE 일반화 우위는 아직 입증하지 못했다는 것이다.

향후 개선 계획은 다음과 같다.

1. 배포된 target-blind union과 관측 가능한 evidence 기반 선택 router를 동일한 새 CVE 집합에서 비교한다.
2. Taint 14 CWE의 FN을 path/import/rule coverage로 분해하고 재현율을 개선하되 FPR 증가 상한을 둔다.
3. Qwen sanitizer와 propagation 후보를 buggy/fixed 쌍에서 검증하고 good-side regression이 없는 규칙만 승격한다.
4. Critic ablation을 수행해 추가 토큰 대비 FP 감소와 F1 개선이 유의한 경우에만 도입한다.
5. 기존 개발 자료와 저장소 계보가 겹치지 않는 재현 가능한 Java CVE를 사전 고정하고 완주한 결과만 보고한다.
6. Provenance, golden, buggy/fixed, holdout, 승인 기록을 연결하는 promotion registry를 자동화한다.

부록 A. 프롬프트 및 규칙 검증 요약

표 21. 서로 다른 두 Qwen 경로의 계약

항목	API 규칙 생성	배포 의미 검토
입력	후보 ID, exact FQN 관련 사실, signature, 호출 문맥, catalogue	원시 Java 파일의 모든 문자, target CWE 없음
출력	role, category, CWE, endpoint, confidence, 이유	CWE, 원본 행, confidence, 근거
안전장치	unresolved이면 none, 내부 parameter는 source/none, exact FQN 재부착	최대 12건, 행 범위·CWE schema 검사, high만 수용
판정 영향	Juliet enforce 실험; 배포는 비활성·shadow	배포 union에 직접 추가, CPG 경고 veto 없음

규칙 생성 실험은 high-confidence exact source/sink만 적용하고 sanitizer, propagation, critic을 제외했다. 이는 여러 LLM 기능을 동시에 켜 성능 원인을 알 수 없게 되는 것을 피하기 위한 통제다. 배포 의미 검토는 24,000자 창과 최대 1,000자 중첩을 사용하며, 일부 창 실패 시 전체 파일을 안전으로 판정하지 않고 PARTIAL로 전파한다.

부록 B. 재현 경로와 공유 코드 범위

현재 공유된 scanops-model main에서 직접 확인되는 분석·평가 코드는 다음과 같다. 경로는 모델 저장소 루트 기준이다.

- 38 CWE 일괄 실행: benchmarks/juliet-java/run_all_cwes.sh
- finding 단위 채점: benchmarks/juliet-java/grade_instances.py
- CPG 후보 구성·규칙 변환: scanops/core/graph_spec_prod.py
- 배포 의미 검토·경고 병합: scanops/core/java_semantic.py
- Java 분석 API: scripts/api_rebuild.py

534쌍 최종 비교에 사용한 economy 계열 manifest·실행·집계 파일은 실험 당시 별도 산출물로 관리되었으며 현재 공유된 GitHub main에는 포함되어 있지 않다. 아래는 실험 당시 기록된 주요 경로이며, 공개 저장소에서 모두 찾을 수 있다는 뜻은 아니다.

- 확정 표본: benchmarks/juliet-java/economy_manifest_v3.json
- 표본 생성: benchmarks/juliet-java/build_economy_manifest.py
- 선택·증립화: benchmarks/juliet-java/juliet_economy.py
- 구현 범위 분류: benchmarks/juliet-java/capability_strata_v1.json
- 정적 경고 채점: benchmarks/juliet-java/score_economy_static.py
- 소스 단독 비교군: benchmarks/juliet-java/run_economy_source_only.py
- 규칙 생성 비교군: benchmarks/juliet-java/run_economy_hybrid_rulegen.py
- Joern 적용 비교군: benchmarks/juliet-java/run_economy_hybrid_taint.py
- 최종 결과: rebuild/out/java_cpg_iris_v2_20260907/JULIET_ECONOMY_FINAL_RESULTS.json

실험 당시 절차는 (1) Juliet 원본 준비와 표본 목록 생성, (2) 목록의 hash·쌍·분할 검증, (3) 동일 split의 네 비교군 실행, (4) CWE·메서드 구간 기준의 채점, (5) 동일 manifest hash와 split의 결과 집계 순서였다. 현재 공개된 저장소만으로는 위 최종 비교를 완전히 재실행할 수 없다. 전체 재현에는 별도 보관된 평가 스크립트·확정 표본·채점 결과가 함께 필요하다. 저장된 최종 JSON의 TP·FP·FN·TN으로 지표를 다시 계산하는 것과 유료 LLM 호출을 포함한 탐지 재실행은 구분한다.

사이트 연결 확인 자료는 로컬의 다음 위치에 보관하였다.

- 폴더: deployment-evidence/20260908/
- 연결 기록: site-java-integration-verification.txt
- 배포 버전: release-version-summary.txt

이 기록은 서비스 연동과 배포 버전을 확인하는 자료이며 Juliet 정량 성능 자료와는 별개다.

2026년 9월 14일에는 Java 엔진과 기존 파인튜닝 모델의 코드·안내를 각 저장소 main에 통합하였다. 당시 기록상 모델 회귀 테스트는 58개와 5개 하위 테스트, 백엔드 테스트는 40개가 통과했다. 이는 9월 8일의 배포 시험과 다른 시점의 소스 통합 검증이며 서버 재배포나 새로운 성능 실험은 수행하지 않았다. 본 보고서의 파일 설명은 로컬 Git에 보존된 모델 main 커밋 32f1bcc0b703와 백엔드 main 커밋 2d47d2971bac를 대조하였다.

부록 C. 배포 버전과 응답 계약

시험 배포에 사용한 원격 커밋의 앞 12자리는 model 61ce3a835105, backend d875279466b0, infra cd09fd871506이다. 전체 해시는 배포 증거의 release-version-summary.txt에 고정했다. FastAPI는 status, source, line, evidence, findings, analysis_details를 반환한다. Spring 백엔드는 Java 요청에 별도 private URL과 X-API-Key를 사용하고 DONE이 아니거나 요청 파일이 누락된 응답을 거절한다. 프론트엔드는 구조 경고와 의미 경고의 source를 구분해 표시한다.

부록 D. 제출 코드와 평가 자료 찾아보기

저장소별 주요 파일

제출 코드는 모델, 백엔드, 프론트엔드, 인프라의 네 저장소로 나뉜다. 모델 저장소의 README.md는 현재 Java 엔진과 과거 파인튜닝 모델을 구분하며, docs/CPG_LLM.md는 현재 실행 방법, docs/VERIFICATION.md는 검증 범위를 설명한다. 보고서는 개발 목적과 구현 근거를 설명하고, README는 설치·실행·결과 확인 절차를 안내한다. 아래 경로는 각 저장소 루트 기준이다.

표 22. 주요 코드 파일의 역할과 입출력

모델 저장소: scanops-model
scripts/api_rebuild.py 분석 API 진입점이다. 언어·파일 경로·소스코드를 받아 Java CPG 분석과 의미 검토를 호출하고, 파일별 상태·취약 위치·근거·복수 경고를 응답으로 구성한다. Java와 기존 비Java 경로를 구분한다.
scanops/core/graph_spec_prod.py CPG 분석의 조정 계층이다. analyze_repo가 저장소 파일과 실행 모드를 받아 Joern 작업과 규칙 구성을 연결한다. build_items_v2는 API 후보와 주변 코드를 입력 항목으로 만들고, label_items_v2, validate, to_tsv가 역할 제안·검증·규칙 변환을 담당한다. 배포 설정에서는 동적 규칙을 판정에 적용하지 않는다.
joern/handler_joern.py 및 joern/queries/dump_candidates_v2.sc 파일 묶음과 분석 모드를 받아 Joern 작업을 실행한다. run_repo_script는 저장소 분석을 처리하고, 후보 추출 질의는 메서드 식별자·호출 위치·파라미터 정보를 반환한다. Joern 자체의 CPG 생성기와 팀이 작성한 실행·질의 연결 코드를 구분한다.
joern/queries/taint_spec.sc 및 scanops/core/specs/java_base.tsv 고정 규칙 및 실험에서 허용된 추가 규칙을 CPG에 적용한다. 입력은 그래프와 규칙이며 출력은 CWE·파일·행·경로 등의 경고 근거다. java_sanitizers.tsv는 고정된 안전 처리 함수 정보를 제공한다.
scanops/core/java_semantic.py review는 Java 파일을 중첩 창으로 검토하고 원본 행 번호를 복원한다. merge는 CPG와 의미 검토 경고를 합친다. 대상 CWE를 입력하지 않으며 high-confidence 의미 경고만 수용한다. LLM 무경고로 CPG 경고를 삭제하지 않는다.
백엔드 저장소: scanops-backend
src/main/java/com/scanops/scan/ScanopsModelClient.java Java/JSP Spring Boot 요청을 별도 분석 API로 라우팅하고 인증 헤더·배치 응답을 처리한다. DONE이 아닌 결과나 요청 파일과 불일치하는 결과를 정상 안전 판정으로 저장하지 않도록 검사한다.

src/main/java/com/scanops/scan/GithubScanService.java 및

src/main/java/com/scanops/scan/GithubPipelineRunner.java

저장소 스캔 작업과 분석 호출·결과 처리를 연결한다. 실험용 채점기와 달리 서비스의 작업 상태 및 사용자에게 반환할 결과를 다루는 계층이다.

프론트엔드 저장소: **scanops-frontend**

src/entities/vulnerability/api/reportApi.ts 및

src/pages/report/ui/ReportPage.tsx

백엔드 보고서 응답을 조회하고 분석 결과를 화면에 표시한다. 탐지 알고리즘의 실행부가 아니라 사용자가 결과를 확인하는 점점이다.

인프라 저장소: **scanops-infra**

docker-compose.java-engine.yml

Java 분석 API와 Joern worker의 실행 구성·연결을 정의한다. 설치·환경변수 준비·기동 절차는 같은 저장소의 docs/JAVA_DEPLOYMENT.md를 따른다.

코드와 평가 자료를 확인하는 순서

실행할 때는 각 저장소 README의 환경변수·설치·기동 절차를 따른다. 구현을 읽을 때는 4장의 처리 흐름과 위 파일 목록을 함께 확인한다. 성능표를 검토할 때는 부록 B의 최종 결과 JSON에서 TP·FP·FN·TN을 확인하고, 각 행의 source 필드를 따라 개별 채점 자료를 확인한다. 공개된 서비스 코드와 별도 보관된 최종 평가 자료의 범위는 부록 B에서 구분하였다.

9. 참고 문헌

- [1] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck, “Modeling and Discovering Vulnerabilities with Code Property Graphs,” in *Proc. IEEE Symposium on Security and Privacy*, 2014.
- [2] Joern Project, “Code Property Graph Documentation,” <https://docs.joern.io/code-property-graph/>, accessed Sep. 2026.
- [3] GitHub, “Analyzing data flow in Java and Kotlin,” <https://codeql.github.com/docs/codeql-language-guides/analyzing-data-flow-in-java/>, accessed Sep. 2026.
- [4] GitHub, “CodeQL query packs,” <https://docs.github.com/en/code-security/concepts/code-scanning/codeql-query-packs>, accessed Sep. 2026.
- [5] Z. Li, S. Dutta, and M. Naik, “LLM-Assisted Static Analysis for Detecting Security Vulnerabilities,” *ICLR*, 2025.
- [6] IRIS Project, “IRIS and CWE-Bench-Java,” <https://github.com/iris-sast/iris>, accessed Sep. 2026.
- [7] C. Wang, W. Zhang, Z. Su, X. Xu, X. Xie, and X. Zhang, “LLMDFA: Analyzing Dataflow in Code with Large Language Models,” *NeurIPS*, 2024.
- [8] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu, “GPTScan: Detecting Logic Vulnerabilities in Smart Contracts by Combining GPT with Program Analysis,” *ICSE*, 2024.
- [9] A. Lekssays, H. Mouhcine, K. Tran, T. Yu, and I. Khalil, “LLMxCPG: Context-Aware Vulnerability Detection Through Code Property Graph-Guided Large Language Models,” arXiv:2507.16585, 2025.
- [10] J. Guo, C. Wang, X. Xu, Z. Su, and X. Zhang, “RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing,” *ICML*, 2025.
- [11] National Institute of Standards and Technology, “Juliet Java 1.3,” Software Assurance Reference Dataset, 2017, <https://samate.nist.gov/SARD/test-suites/111>.
- [12] National Institute of Standards and Technology, “Juliet Test Suite v1.1 for Java User Guide,” SAMATE, 2012, https://samate.nist.gov/SARD/resources/Juliet_Test_Suite_v1.1_for_Java_-_User_Guide.pdf.
- [13] GitHub, “CodeQL Queries,” <https://codeql.github.com/docs/writing-codeql-queries/codeql-queries/>, accessed Sep. 2026.
- [14] TUHH SoftSec, “Vul4J: A Dataset of Reproducible Java Vulnerabilities,” <https://github.com/tuhh-softsec/vul4j>, accessed Sep. 2026.